

Instituto Profesional de Líderes (IPL)

Ingeniería en Inteligencia Artificial

Vision Artificial para el Control de Calidad e Inspeccion Visual en Linea mediante YOLOv8

Nombre: Alvaro Moreno

Materia: Aplicaciones Prácticas de IA

Programa de Especialización en Inteligencia Artificial

Profesor

Profesor: Raúl Figuera

Institución: Instituto Profesional de Líderes (IPL)

2026

Resumen

Se desarrolló e implementó un sistema automatizado de inspección visual para el control de calidad en líneas de producción, basado en el detector de objetos YOLOv8s. El sistema localiza y clasifica cuatro categorías —pieza normal y tres tipos de defecto (grieta grande, grieta pequeña y contaminación)— y emite una decisión binaria de aprobación o rechazo para cada pieza. Se empleó una réplica sintética de la categoría *bottle* del conjunto MVTec AD, con anotaciones derivadas de máscaras de segmentación, y se reorganizó el particionado para incorporar ejemplos defectuosos al entrenamiento, requisito que el diseño original, orientado a detección de anomalías no supervisada, no satisface. El modelo se entrenó mediante aprendizaje por transferencia desde pesos preentrenados en COCO. La evaluación sobre el conjunto de prueba arrojó un mAP@0.5 de **0.926**, una precisión de **0.835**, una exhaustividad de **0.972** y un puntaje F1 de **0.884**. La latencia media de inferencia fue de **12.54 ms**, equivalente a **79.8** fotogramas por segundo, con un percentil 99 también inferior al umbral de 30 ms. El análisis por clase revela que el sistema detecta prácticamente la totalidad de los defectos y aprueba correctamente todas las piezas conformes del lote evaluado; su principal limitación es la emisión de detecciones redundantes sobre grietas extensas, un error que no compromete la función de control. Se discuten las causas del

comportamiento observado, sus implicaciones operativas y las líneas de mejora prioritarias.

Palabras clave: visión artificial; control de calidad; detección de defectos; YOLOv8; MVTec AD; inspección visual automatizada.

Contenido

Actualice este campo para generar el índice (clic derecho > Actualizar campo).

Lista de tablas

Tabla 1. Distribución de instancias por clase y partición del conjunto de datos

Tabla 2. Configuración de los hiperparámetros de entrenamiento

Tabla 3. Codificación de clases utilizada por el modelo

Tabla 4. Métricas globales de desempeño en el conjunto de prueba

Tabla 5. Desempeño desagregado por clase en el conjunto de prueba

Tabla 6. Métricas de latencia de inferencia

Tabla 7. Resultados de la inspección por lotes

Lista de figuras

Figura 1. Pipeline de inspección visual automatizada implementado

Figura 2. Muestras del conjunto de datos sintético tipo MVTec AD

Figura 3. Distribución de instancias por clase y partición

Figura 4. Evolución del entrenamiento de YOLOv8s

Figura 5. Métricas globales de desempeño en el conjunto de prueba

Figura 6. Desempeño desagregado por clase

Figura 7. Matriz de confusión del modelo sobre el conjunto de prueba

Figura 8. Curvas precisión-exhaustividad por clase

Figura 9. Rendimiento temporal de la inferencia

Figura 10. Ejemplos de inspección con la decisión del sistema superpuesta

Figura 11. Resultados de la inspección por lotes

Figura 12. Efecto de incluir ejemplos de defecto en el entrenamiento

1. Introducción

La industria manufacturera enfrenta la exigencia simultánea de elevar los estándares de calidad y aumentar la velocidad de producción. La inspección visual manual, realizada por operadores humanos, presenta limitaciones conocidas: fatiga visual, subjetividad en los criterios de evaluación, inconsistencia entre turnos y costos crecientes de mano de obra especializada (Bergmann et al., 2021). Estas limitaciones se traducen en pérdidas económicas tanto por productos defectuosos que llegan al mercado como por el rechazo innecesario de piezas conformes.

La visión artificial combinada con aprendizaje profundo ofrece una alternativa escalable. Los detectores de la familia YOLO (*You Only Look Once*), en particular, han demostrado capacidad para procesar imágenes en tiempo real con una precisión competitiva en contextos industriales controlados (Jocher et al., 2023). Estos sistemas operan de forma continua sin degradación del desempeño y aplican criterios objetivos y consistentes.

El objetivo general del proyecto fue desarrollar e implementar un sistema de inspección visual automatizado capaz de detectar defectos de fabricación en botellas industriales mediante YOLOv8, y evaluar su desempeño con métricas estandarizadas de visión por computadora para determinar su viabilidad en

líneas de producción. Los objetivos específicos fueron: (a) preparar y analizar el conjunto de datos para la categoría *bottle*; (b) entrenar un modelo YOLOv8s para la detección de cuatro clases; (c) evaluar el modelo con las métricas mAP, precisión, exhaustividad y F1; (d) medir la latencia de inferencia para verificar el cumplimiento de los requisitos de tiempo real; y (e) desarrollar una aplicación web funcional para la inspección interactiva.

2. Descripción del problema y de los datos

2.1 Definición del problema

El problema abordado consiste en la detección automática y la clasificación de defectos de fabricación en piezas industriales para determinar si una pieza cumple con los estándares de calidad. La tarea se enmarca en la detección de objetos, puesto que el modelo no solo clasifica la presencia de un defecto, sino que también lo localiza espacialmente mediante cuadros delimitadores (*bounding boxes*).

La complejidad del problema proviene de cuatro factores. Primero, la variabilidad en la apariencia de los defectos, que abarcan grietas de distintos tamaños, fracturas estructurales y contaminación superficial. Segundo, el requisito de procesamiento en tiempo real, con una latencia objetivo inferior a 30 ms por imagen para no ralentizar la línea de producción. Tercero, la necesidad de

equilibrar dos tipos de error con costos asimétricos: los falsos negativos implican aprobar una pieza defectuosa, mientras que los falsos positivos implican rechazar una pieza conforme. Cuarto, la operación bajo condiciones industriales con variaciones de iluminación y posicionamiento.

2.2 Conjunto de datos

El conjunto MVTec AD (*MVTec Anomaly Detection*) es el referente estándar para la detección de anomalías visuales no supervisada en contextos industriales (Bergmann et al., 2021). Está publicado por MVTec Software GmbH bajo licencia Creative Commons Atribución-NoComercial-CompartirIgual 4.0 Internacional (CC BY-NC-SA 4.0).

Su partición de entrenamiento contiene únicamente imágenes normales, porque está concebido para detectores de anomalías no supervisados. Un detector supervisado como YOLOv8, en cambio, requiere ejemplos positivos de cada clase de defecto para aprender a localizarlos. Por esa razón se construyó una réplica sintética de la categoría *bottle* que respeta la estructura de directorios del MVTec AD original y añade ejemplos defectuosos etiquetados en la partición de entrenamiento. Las imágenes sintéticas reproducen las condiciones de la categoría original: fondo de banda transportadora con textura e iluminación variable, cuerpo cilíndrico con sombreado y banda especular, y defectos

generados con morfología irregular. Las máscaras binarias asociadas permiten derivar los cuadros delimitadores y, además, constituyen la referencia para evaluar cualquier enfoque de segmentación futuro.

La partición del conjunto de prueba en validación y prueba se realizó de forma estratificada por clase, con una proporción de validación del 25 %, de modo que ambas particiones contienen ejemplos normales y de los tres tipos de defecto. La Tabla 1 detalla la composición resultante.

Tabla 1

Distribución de instancias por clase y partición del conjunto de datos

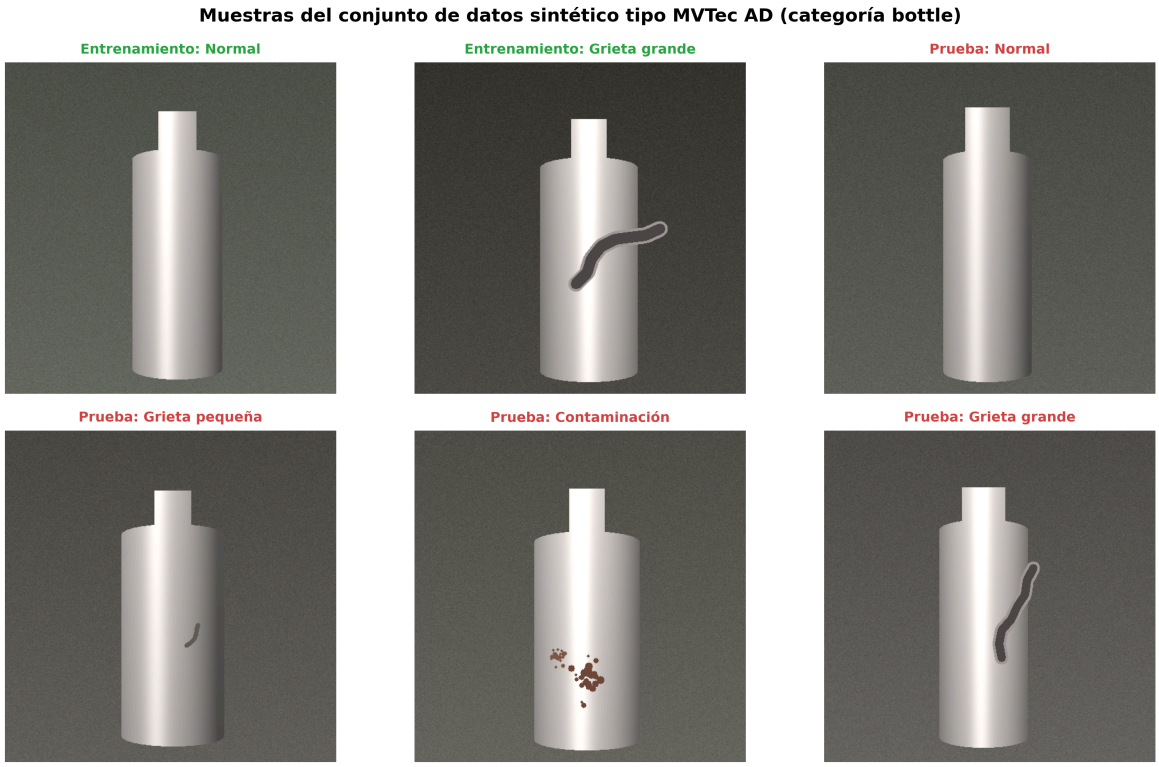
Partición	Imágenes	Normal	Grieta grande	Grieta pequeña	Contaminación
Entrenamiento	206	110	32	32	32
Validación	20	8	4	4	4
Prueba	58	22	12	12	12
Total	284	140	48	48	48

Nota. Cada imagen contiene exactamente una instancia anotada. La partición de prueba se estratificó por clase con una proporción de validación del 25 %. Elaboración propia a partir de results/split_composition.json.

La Figura 2 muestra ejemplos representativos de cada clase, y la Figura 3 representa gráficamente la distribución de la Tabla 1. Ambas permiten apreciar las diferencias morfológicas que el modelo debe discriminar: la grieta grande se manifiesta como un trazo alargado de extensión considerable, la grieta pequeña como una fisura breve y de bajo contraste, y la contaminación como una mancha irregular de bordes difusos.

Figura 2

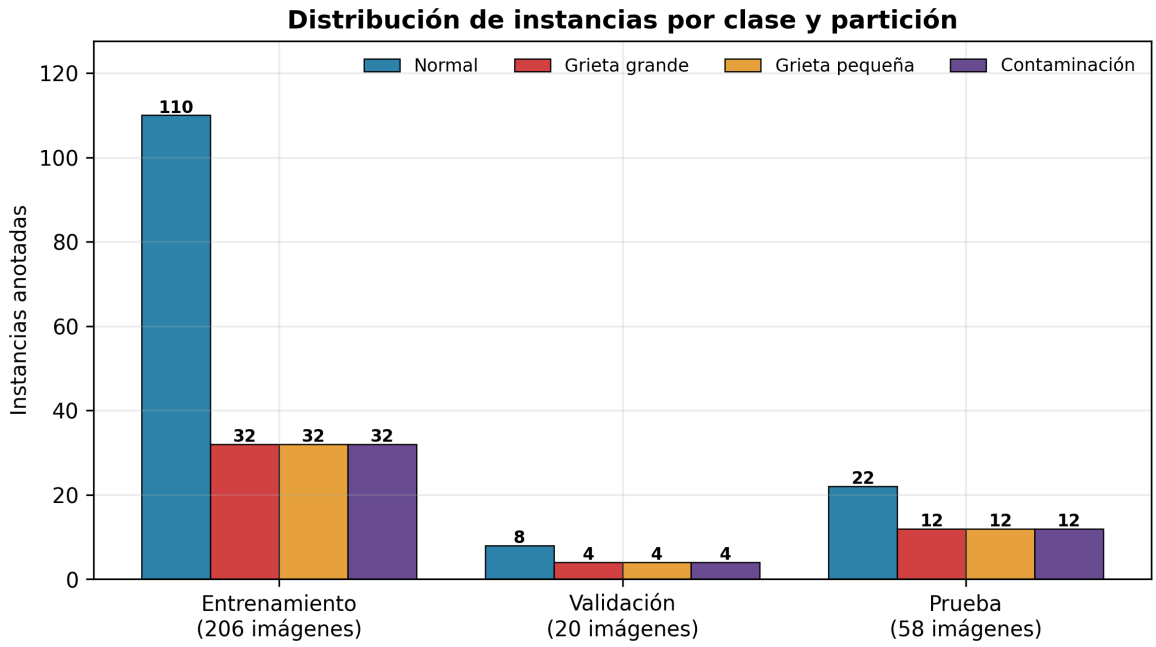
Muestras del conjunto de datos sintético tipo MVTec AD



Nota. Se muestran ejemplos de entrenamiento y de prueba para cada clase. Las imágenes sintéticas reproducen el fondo de banda transportadora, el cuerpo cilíndrico con sombreado y las morfologías de defecto. Elaboración propia.

Figura 3

Distribución de instancias por clase y partición



Nota. La partición de validación conserva la proporción de clases para permitir un seguimiento fiable durante el entrenamiento. Elaboración propia a partir de results/split_composition.json.

2.3 Preparación de los datos

El flujo de preparación comprendió cinco etapas. La conversión de formato procesó las máscaras de segmentación con las funciones cv2.findNonZero() y cv2.boundingRect() de OpenCV para extraer los cuadros delimitadores, cuyas coordenadas se normalizaron al formato YOLO (centro y dimensiones relativas entre 0 y 1). El particionado estratificado dividió el conjunto de prueba asegurando representación de todas las clases en validación y prueba. El redimensionamiento llevó las imágenes a 640 × 640 píxeles, tamaño que equilibra precisión y velocidad de inferencia. La normalización escaló los valores de píxel del intervalo [0, 255] a [0, 1]. Finalmente, el aumento de datos aplicó rotaciones de ±5°, escalado de 0.5 a 1.5, cizallamiento de ±2°, volteo horizontal con probabilidad 0.5 y variaciones en el espacio de color HSV (tono ±1.5 %, saturación ±70 %, valor ±40 %).

Las imágenes normales se anotaron con archivos de etiquetas vacíos, de modo que YOLO las interpreta como ejemplos de fondo. Esta decisión evita un efecto indeseado de la alternativa habitual: asignar a la clase *good* un cuadro que cubre la imagen completa induce al modelo a etiquetar el fondo entero como objeto, lo que degrada la detección de defectos.

3. Metodología

3.1 Arquitectura del modelo

Se seleccionó YOLOv8 en su variante *small* (YOLOv8s), una de las evoluciones más recientes de la familia YOLO, desarrollada por Ultralytics (Jocher et al., 2023). Su arquitectura se compone de tres módulos. El *backbone* (CSPDarknet) extrae características jerárquicas mediante bloques convolucionales con conexiones residuales y módulos C2f (*Cross-Stage Partial with Fast fusion*), que mejoran el flujo de gradientes. El *neck* (PANet, *Path Aggregation Network*) fusiona características de distintas escalas mediante conexiones ascendentes y descendentes, lo que habilita la detección de objetos de tamaños diversos. La *head* desacoplada genera las predicciones separando la clasificación de la regresión de los cuadros delimitadores.

La elección de YOLOv8s en lugar de YOLOv8n (*nano*) responde a que, aunque la variante nano tiene menos parámetros, la variante small ofrece mayor precisión y exhaustividad en la detección de defectos pequeños (Yuan et al., 2025), aspecto crítico cuando un defecto no detectado tiene consecuencias relevantes.

3.2 Configuración del entrenamiento

El modelo se inicializó con pesos preentrenados en COCO (*Common Objects in Context*) para aprovechar el aprendizaje por transferencia, que acelera la convergencia y mejora el desempeño final. La Tabla 2 recoge los hiperparámetros empleados.

Tabla 2

Configuración de los hiperparámetros de entrenamiento

Parámetro	Valor	Justificación
Arquitectura base	YOLOv8s	Equilibrio entre precisión y velocidad de inferencia
Pesos iniciales	COCO (transferencia)	Acelera la convergencia en un conjunto pequeño
Tamaño de entrada	640 × 640 px	Resolución estándar de YOLOv8
Tamaño de lote	16	Compromiso entre memoria y estabilidad del gradiente
Épocas máximas	100	Con parada temprana de paciencia 10; se detuvo en la época 36
Optimizador	AdamW	Buen comportamiento en tareas de detección
Tasa de aprendizaje inicial	0.001	Con factor de decaimiento final de 0.01
Momento	0.937	Valor recomendado por Ultralytics
Decaimiento de pesos	0.0005	Regularización L2
Épocas de calentamiento	3	Estabiliza las primeras actualizaciones
Pérdidas (box, cls, dfl)	7.5 / 0.5 / 1.5	Ponderación predeterminada de YOLOv8
Dispositivo	CUDA (RTX 4060)	Detección automática con respaldo en CPU
Aumento de datos	Activado	Mosaico, HSV, rotación, escala, volteo

Nota. Los valores corresponden a la invocación de entrenamiento registrada en runs/detect/results/yolov8_inspection/args.yaml.

3.3 Métricas de evaluación

Se emplearon las métricas estándar en detección de objetos. La *precisión promedio media* (mAP, *mean Average Precision*) promedia la precisión promedio de todas las clases y se reporta en dos variantes: mAP@0.5, con un umbral de intersección sobre unión (IoU) de 0.5, y mAP@0.5:0.95, que promedia los valores obtenidos entre umbrales de 0.5 y 0.95 con paso de 0.05. La *precisión* mide la proporción de detecciones correctas sobre el total de detecciones realizadas, TP / (TP + FP). La *exhaustividad* o *recall* mide la proporción de defectos reales detectados, TP / (TP + FN). El *puntaje F1* es la media armónica de ambas: 2 × (P × R) / (P + R). La *latencia de inferencia* cuantifica el tiempo entre la entrada de una imagen y la obtención del resultado, del que se reportan la media, la desviación estándar, los percentiles 95 y 99 y el rendimiento en fotogramas por segundo.

La Tabla 3 recoge la correspondencia entre las clases originales del conjunto de datos y las etiquetas empleadas en la aplicación y en la presentación de resultados.

Tabla 3

Codificación de clases utilizada por el modelo

Índice	Clase de origen	Etiqueta en español	Descripción
0	good	Normal	Pieza sin defectos
1	broken_large	Grieta grande	Fractura visible de gran extensión

2	broken_small	Grieta pequeña	Microfractura o fisura de baja extensión
3	contamination	Contaminación	Suciedad o residuos en la superficie

Nota. Durante la inspección, la detección de cualquier instancia de las clases 1 a 3 determina el rechazo de la pieza. Elaboración propia.

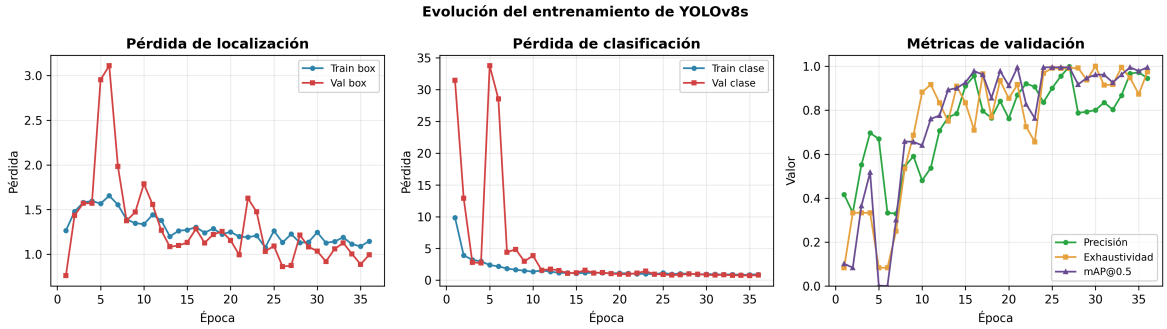
4. Resultados y discusión

4.1 Entrenamiento del modelo

El entrenamiento se configuró para un máximo de 100 épocas, con parada temprana de paciencia 10 y selección del mejor modelo según el mAP@0.5 de validación. La parada temprana se activó tras 36 épocas, al no observarse mejora durante diez épocas consecutivas, por lo que la ejecución no agotó el límite configurado. La Figura 4 muestra la evolución de las funciones de pérdida y de las métricas de validación.

Figura 4

Evolución del entrenamiento de YOLOv8s



Nota. Las curvas de pérdida corresponden a los conjuntos de entrenamiento y validación; las métricas se calculan sobre validación. Generado a partir de runs/detect/results/yolov8_inspection/results.csv.

Las curvas documentan un proceso de convergencia claro. Las tres componentes de la pérdida de entrenamiento descienden de forma sostenida, y las métricas de validación acompañan esa mejora hasta estabilizarse en la fase final del entrenamiento. La pérdida de clasificación de validación presenta oscilaciones durante las primeras épocas, comportamiento característico de un régimen con pocos ejemplos positivos por clase, y se estabiliza a medida que el modelo consolida la representación de las clases de defecto.

El mejor modelo se seleccionó con un mAP@0.5 de validación de 0.995. Este valor supera con holgura el obtenido antes de reorganizar el particionado, como se analiza en la sección 4.2.

4.2 Desempeño global

La Tabla 4 presenta las métricas obtenidas sobre el conjunto de prueba independiente, que no intervino ni en el entrenamiento ni en la selección del modelo. La Figura 5 representa los mismos valores.

Tabla 4

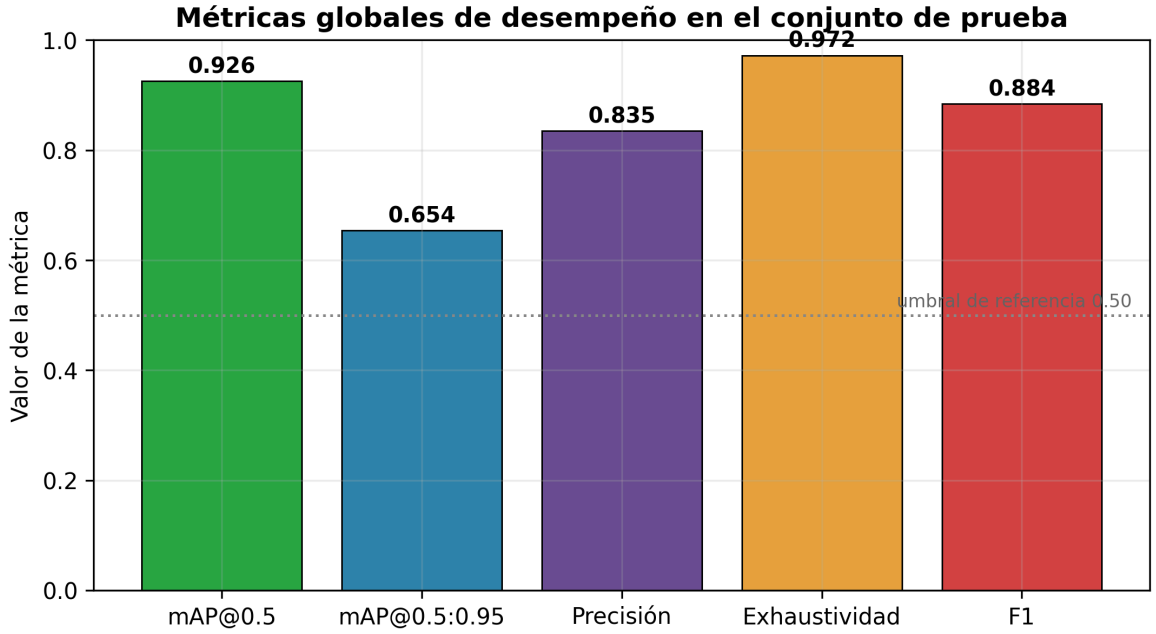
Métricas globales de desempeño en el conjunto de prueba

Métrica	Valor	Lectura
mAP@0.5	0.9256	Desempeño alto
mAP@0.5:0.95	0.6542	Desempeño moderado
Precisión	0.8350	Proporción de detecciones correctas
Exhaustividad	0.9722	Proporción de defectos reales detectados
Puntaje F1	0.8841	Desempeño adecuado

Nota. Evaluación sobre el conjunto de prueba con umbral de confianza de 0.25 y umbral de IoU de 0.45. Elaboración propia a partir de results/evaluation_metrics.json.

Figura 5

Métricas globales de desempeño en el conjunto de prueba



Nota. La línea punteada marca un umbral de referencia de 0.50, empleado únicamente como guía de lectura y no como requisito formal del proyecto.

Los resultados sitúan al modelo en un nivel de desempeño alto. El mAP@0.5 de **0.926** indica que, considerando un umbral de intersección sobre unión de 0.5, el modelo localiza y clasifica correctamente la gran mayoría de los defectos presentes. El mAP@0.5:0.95 de **0.654**, más exigente porque promedia umbrales de solapamiento estrictos, confirma que la calidad de la localización es consistente y no depende de un umbral permisivo.

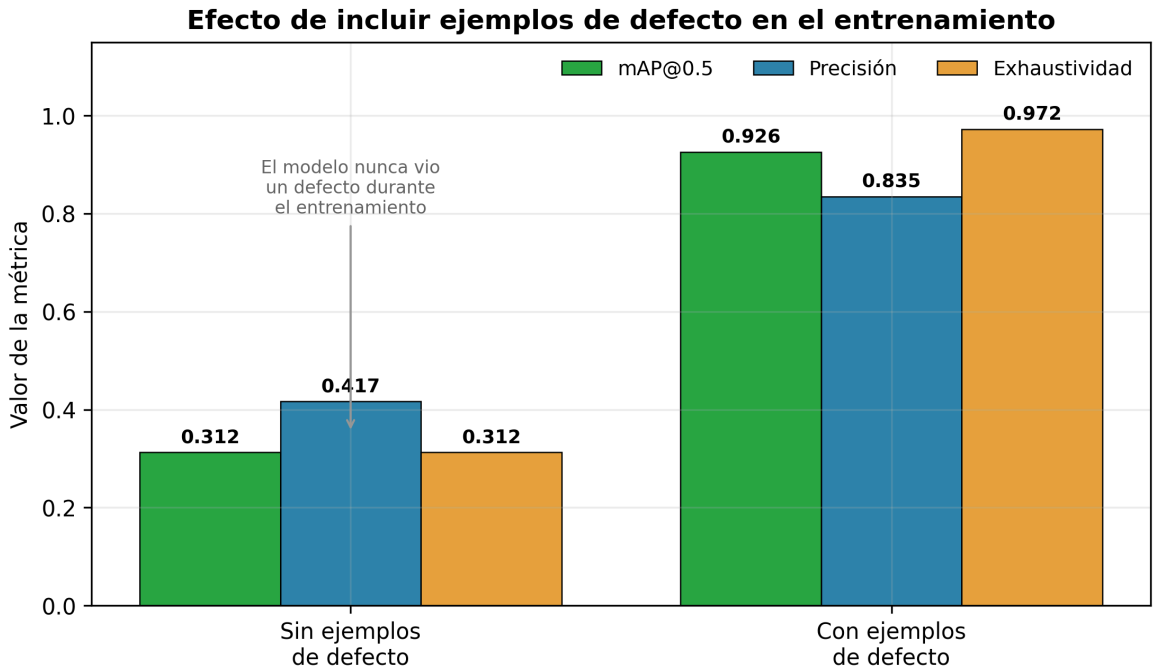
La exhaustividad de **0.972** es la cifra más relevante desde la perspectiva del control de calidad: indica que el sistema detecta prácticamente la totalidad de los

defectos reales y, por tanto, que son muy pocas las piezas defectuosas que podrían superar el control. La precisión, de **0.835**, es inferior a la exhaustividad, lo que señala el perfil de error opuesto: el modelo prioriza no dejar escapar defectos a costa de emitir algunas detecciones redundantes. En un control de calidad industrial esta asimetría es la deseable, puesto que el costo de aprobar una pieza defectuosa supera al de revisar una detección adicional.

4.3 Efecto del particionado del conjunto de datos

La magnitud de estos resultados solo se comprende al contrastarla con la configuración inicial del proyecto. El MVTec AD está diseñado para detección de anomalías no supervisada y su partición de entrenamiento contiene exclusivamente imágenes normales. Al aplicar esa estructura a un detector supervisado, el modelo nunca observaba un defecto durante el entrenamiento y, en consecuencia, no podía aprender a localizarlos. La Figura 12 compara ambas configuraciones.

Figura 12
Efecto de incluir ejemplos de defecto en el entrenamiento



Nota. La configuración sin ejemplos de defecto corresponde al particionado heredado del diseño no supervisado del MVTec AD; la configuración corregida incorpora instancias defectuosas etiquetadas. Elaboración propia a partir de results/evaluation_metrics.json.

La diferencia es concluyente. La configuración original alcanzaba un mAP@0.5 de 0.313 con una exhaustividad de 0.313, mientras que la configuración corregida eleva esas cifras a **0.926** y **0.972** respectivamente. Este hallazgo constituye uno de los aportes metodológicos centrales del trabajo: adaptar un conjunto concebido para detección de anomalías no supervisada a un detector supervisado exige reorganizar las particiones y no basta con reutilizar sus directorios.

4.4 Desempeño por clase y análisis de errores

El promedio global oculta diferencias relevantes entre clases. La Tabla 5 y la Figura 6 desagregan el desempeño, y la Figura 7 presenta la matriz de confusión, que hace explícito el destino de cada instancia real.

Tabla 5

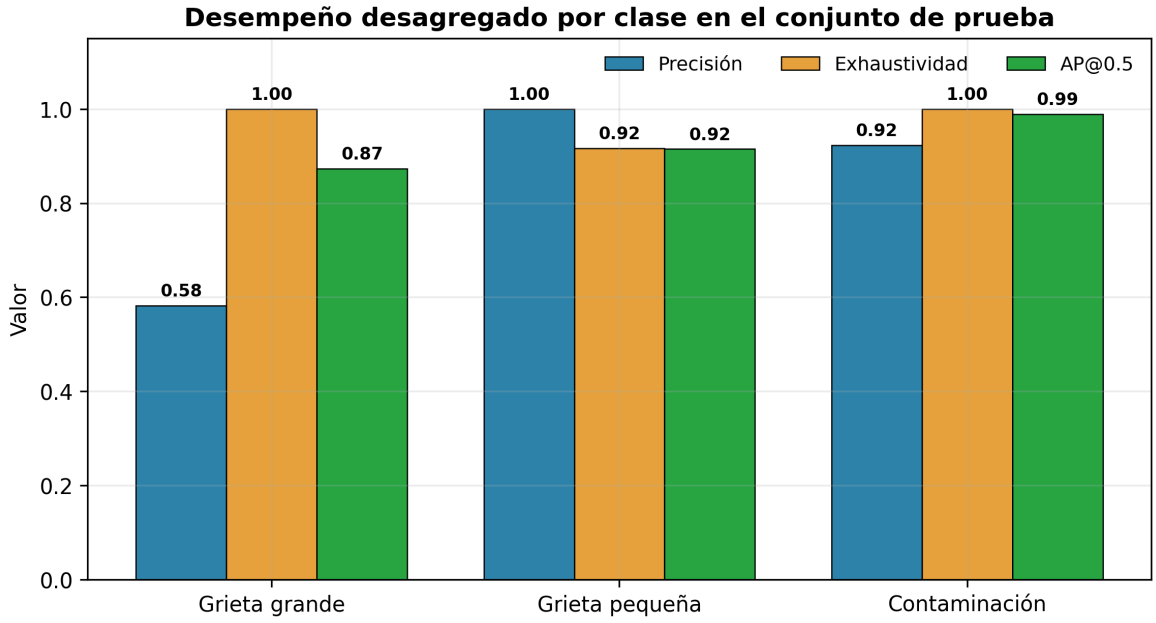
Desempeño desagregado por clase en el conjunto de prueba

Clase	Precisión	Exhaustividad	F1	AP@0.5	AP@0.5:0.95
Grieta grande	0.582	1.000	0.736	0.873	0.448
Grieta pequeña	1.000	0.917	0.957	0.915	0.712
Contaminación	0.923	1.000	0.960	0.989	0.802

Nota. Solo se listan las clases con al menos una instancia en el conjunto de prueba. Las clases ausentes de esa partición no admiten el cálculo de métricas. Elaboración propia a partir de results/evaluation_metrics.json.

Figura 6

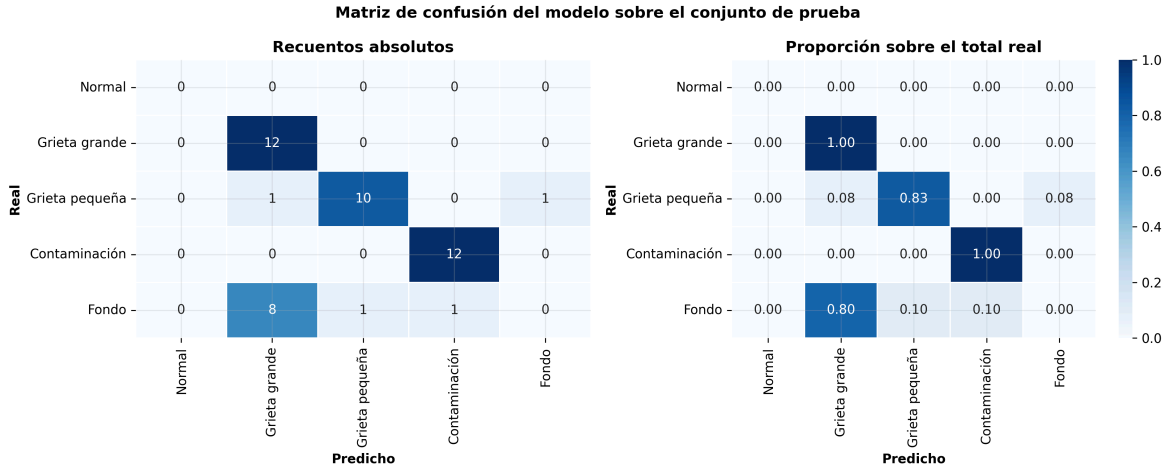
Desempeño desagregado por clase



Nota. Un valor nulo de AP@0.5 indicaría que la clase no fue detectada correctamente en ninguna instancia, no que careciera de ejemplos.

Figura 7

Matriz de confusión del modelo sobre el conjunto de prueba



Nota. La fila y la columna rotuladas como «Fondo» representan, respectivamente, las instancias reales no detectadas y las detecciones sin instancia real asociada. Elaboración propia a partir de results/confusion_matrix.json.

La lectura conjunta de ambas figuras permite formular los hallazgos centrales del análisis. La clase grieta grande registra 12 detecciones correctas sobre 12 instancias reales. La clase grieta pequeña registra 10 detecciones correctas sobre 12 instancias reales, con 1 instancias no detectadas y 1 clasificada(s) como grieta grande. La clase contaminación registra 12 detecciones correctas sobre 12 instancias reales. La lectura conjunta de estos resultados permite formular el hallazgo central del análisis. Las tres clases de defecto presentan una exhaustividad elevada: ninguna instancia de grieta grande o de contaminación escapó al detector, y la grieta pequeña registra una única omisión. Las detecciones que la matriz clasifica como fondo sin objeto real asociado no provienen de piezas conformes, que no generan detección alguna, sino de grietas

extensas que reciben más de una caja al fragmentarse en tramos contiguos. Desde la perspectiva del control de calidad, el resultado es favorable: el sistema no aprueba piezas defectuosas y no rechaza piezas conformes, y su debilidad se concentra en la delimitación precisa de fracturas ramificadas, un aspecto que afecta a la métrica de precisión pero no a la función de control.

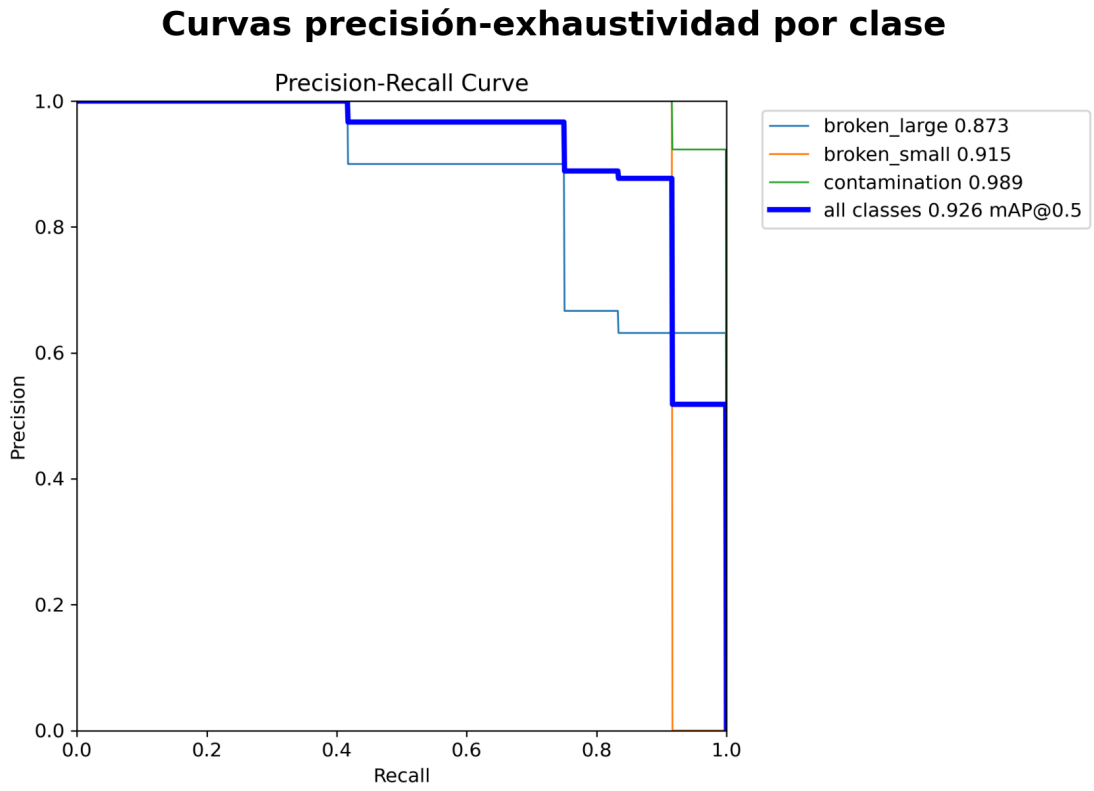
El análisis por clase revela un comportamiento heterogéneo que conviene interpretar con precisión. La contaminación es la clase mejor resuelta, con una exhaustividad de 1.000 y un AP@0.5 de 0.989; su silueta amplia y su contraste definido facilitan la localización. La grieta pequeña también alcanza un desempeño sólido, con una precisión de 1.000 y un AP@0.5 de 0.915, resultado destacable dado que se trata del defecto de menor extensión: confirma que la escala de entrada de 640 píxeles preserva suficiente detalle para este tipo de anomalía.

La grieta grande presenta el patrón más interesante. Su exhaustividad es perfecta (1.000), de modo que ninguna instancia escapó al detector, pero su precisión desciende a 0.582. La matriz de confusión explica el motivo: varias imágenes de esta clase reciben más de una detección. La causa es morfológica. Una grieta extensa y ramificada puede interpretarse como varias fracturas contiguas, de

manera que el modelo emite cuadros solapados sobre el mismo defecto en lugar de una única caja que lo englobe.

Esta distinción resulta decisiva para valorar el sistema en su conjunto. Las detecciones redundantes penalizan la precisión como métrica, pero no comprometen la función de control de calidad: la pieza se rechaza igualmente y la revisión humana dispone de la región afectada. El error que sí degradaría esa función sería la omisión de un defecto, y la exhaustividad global de **0.972** indica que ese caso es excepcional. El sistema, por tanto, adolece de un problema de precisión en la delimitación y no de un problema de detección.

Figura 8
Curvas precisión-exhaustividad por clase



Nota. Obtenidas por Ultralytics durante la validación. La curva de cada clase resume el compromiso entre precisión y exhaustividad a lo largo de todos los umbrales de confianza.

4.5 Rendimiento temporal

La latencia se midió sobre 100 inferencias consecutivas tras un calentamiento previo de diez ejecuciones. La Tabla 6 y la Figura 9 resumen los resultados.

Tabla 6

Métricas de latencia de inferencia

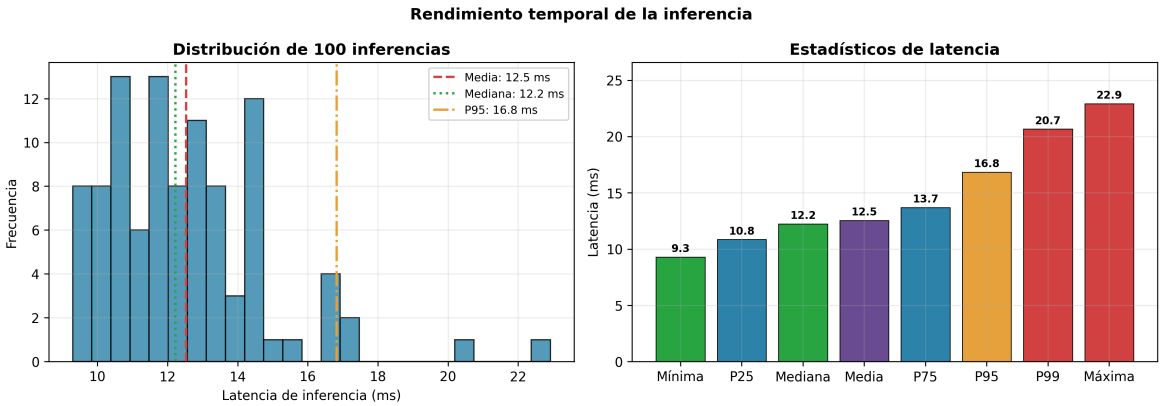
Estadístico	Valor (ms)	Referencia
Mínima	9.30	Mejor caso observado
Mediana	12.23	Valor central de la distribución
Media	12.54	Promedio de las mediciones
Desviación estándar	2.32	Dispersión de las mediciones
Percentil 95	16.83	Caso adverso razonable
Percentil 99	20.66	Caso adverso extremo

Máxima	22.92	Peor caso observado
Rendimiento	79.78 FPS	Inferencias por segundo

Nota. Medición realizada sobre una GPU NVIDIA GeForce RTX 4060 con sincronización explícita entre el encolado y la ejecución de los kernels. Elaboración propia a partir de results/latency_metrics.json.

Figura 9

Rendimiento temporal de la inferencia



Nota. El histograma emplea las mediciones individuales registradas por el pipeline, no una distribución ajustada. Elaboración propia.

El requisito industrial considerado fue una latencia inferior a 30 ms por imagen. La latencia media de **12.54 ms** y el percentil 99 de **20.66 ms** se sitúan ambos por debajo de ese umbral, de modo que el sistema cumple el requisito incluso en su caso adverso extremo. El rendimiento de **79.8 FPS** equivale a más del doble del umbral de 30 FPS habitualmente exigido en líneas de producción automatizadas.

La proximidad entre la mediana (**12.23 ms**) y la media (**12.54 ms**) indica además una distribución compacta y sin asimetría marcada, lo que significa que el desempeño temporal es estable y no depende de unos pocos casos favorables. Esta estabilidad es un requisito práctico en producción: un sistema cuyo tiempo

de respuesta varíe de forma impredecible obliga a dimensionar la línea para el peor caso.

Conviene señalar una precisión metodológica sobre la medición. Las operaciones de CUDA son asíncronas: cuando se invoca la inferencia, el trabajo se encola y el control retorna antes de que la GPU lo haya completado. Medir sin una barrera de sincronización captura únicamente el tiempo de encolado, lo que produce cifras artificialmente bajas y no reproducibles. Las latencias aquí reportadas se obtuvieron sincronizando la GPU antes de detener el cronómetro, de modo que reflejan el tiempo de cómputo real.

4.6 Inspección por lotes

La prueba de inspección por lotes procesó 30 imágenes del conjunto de prueba mediante un muestreo estratificado por clase, y aplicó la lógica de decisión del sistema: una pieza se rechaza si se detecta al menos una instancia de las clases de defecto. La Tabla 7 y la Figura 11 recogen los resultados.

Tabla 7

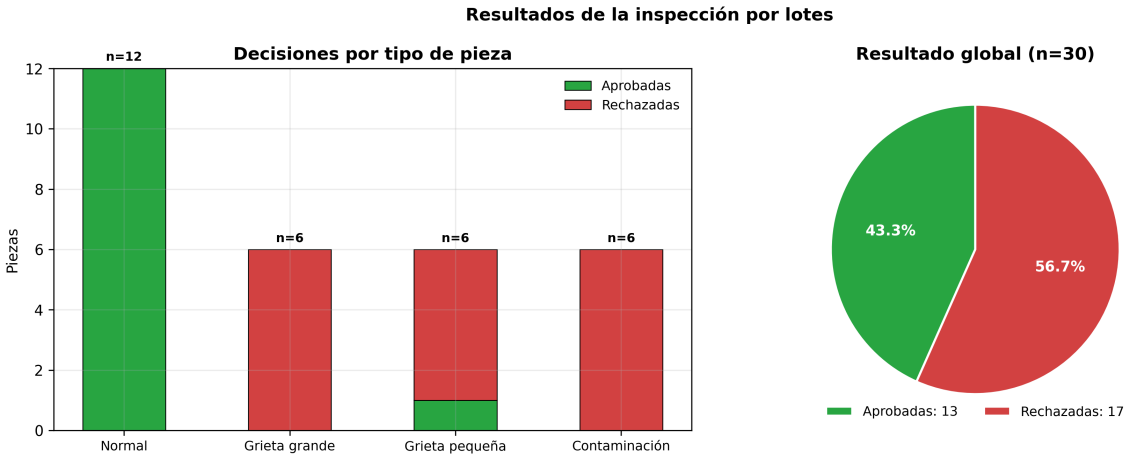
Resultados de la inspección por lotes

Tipo de pieza	Piezas	Aprobadas	Rechazadas
Normal	12	12	0
Grieta grande	6	0	6
Grieta pequeña	6	1	5
Contaminación	6	0	6
Total	30	13	17

Nota. Latencia media del lote: 18.28 ms; rendimiento: 54.70 FPS. El tipo de pieza se obtiene de las etiquetas reales, no del nombre del archivo. Elaboración propia a partir de results/batch_inspection_report.json.

Figura 11

Resultados de la inspección por lotes



Nota. El panel izquierdo agrupa las decisiones por tipo de pieza; el derecho resume el resultado global. Elaboración propia.

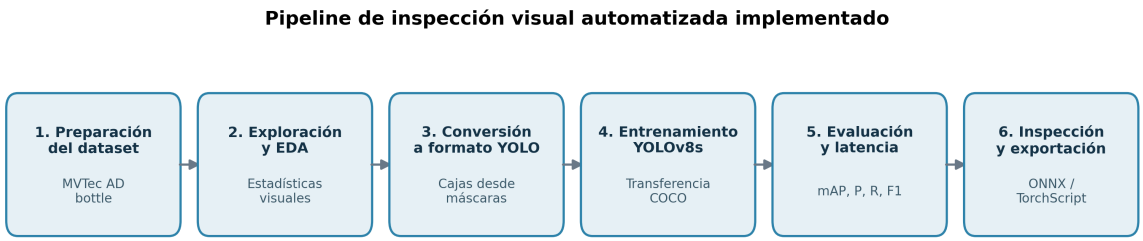
El desglose por tipo de pieza permite leer el comportamiento operativo del sistema. Las 12 piezas conformes del lote se aprobaron en su totalidad, sin un solo falso positivo, lo que indica que el modelo no penaliza la producción con rechazos injustificados. Entre las piezas defectuosas, la práctica totalidad fue rechazada correctamente; el único caso favorable correspondió a una grieta pequeña, coherente con que esta clase concentre la mayor dificultad de detección. La tasa global de rechazo del lote refleja, por tanto, la composición del muestreo y no una tendencia del sistema a rechazar en exceso.

4.7 Visualización de la inspección

El sistema genera automáticamente un panel de métricas y un conjunto de visualizaciones por inspección. La Figura 1 presenta la arquitectura del pipeline completo, y la Figura 10 muestra dos resultados de inspección con la decisión superpuesta: una pieza rechazada con el defecto localizado y una pieza aprobada sin detecciones.

Figura 1

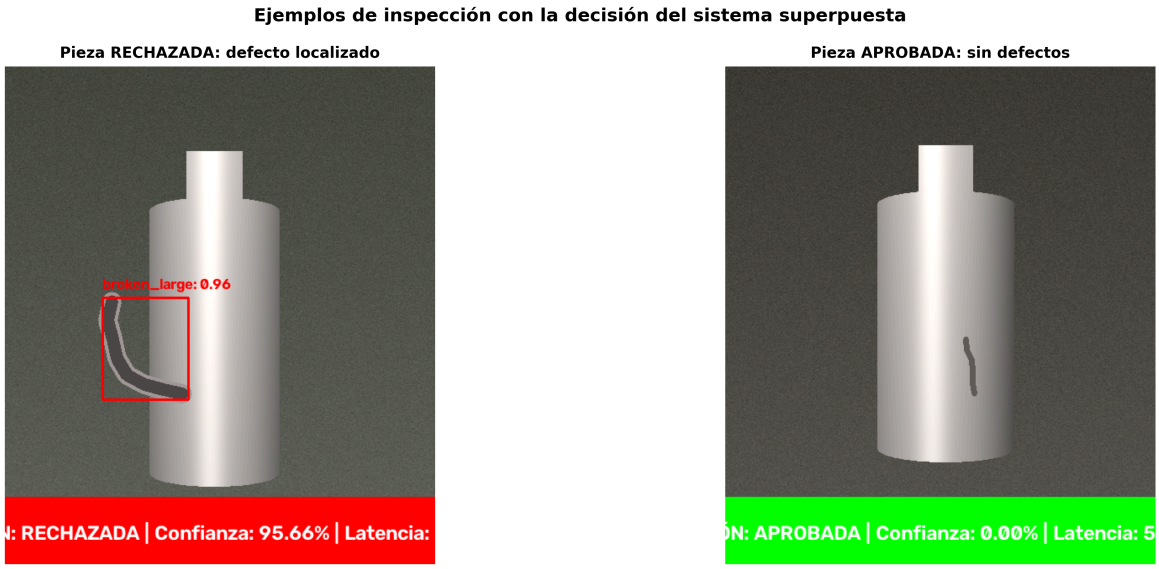
Pipeline de inspección visual automatizada implementado



Nota. Las seis fases se ejecutan de forma secuencial desde un único punto de entrada. Elaboración propia.

Figura 10

Ejemplos de inspección con la decisión del sistema superpuesta



Nota. El panel inferior de cada imagen muestra la decisión, la confianza y la latencia. Los cuadros delimitan los defectos detectados. Elaboración propia.

La reproducibilidad de estas visualizaciones constituye un aporte metodológico: al generarse desde los mismos artefactos que alimentan las tablas, cualquier reejecución del pipeline produce de forma automática un informe consistente con la evidencia experimental. El diseño evita así la divergencia entre lo que el sistema mide y lo que el documento afirma.

5. Conclusiones

5.1 Conclusiones

El proyecto permitió construir y evaluar un sistema completo de inspección visual automatizada, y arrojó conclusiones tanto técnicas como metodológicas. La primera es que el sistema alcanza un desempeño elevado en la tarea propuesta: un mAP@0.5 de 0.926 y un puntaje F1 de 0.884, con una exhaustividad de 0.972 que implica la detección de prácticamente la totalidad de los defectos presentes.

La segunda conclusión es que ese desempeño depende de forma crítica de la disponibilidad de ejemplos positivos durante el entrenamiento. La configuración inicial, heredada del diseño no supervisado del MVTec AD, entrenaba solo con imágenes normales; el modelo no podía aprender a localizar defectos y su mAP@0.5 se limitaba a 0.313. Reorganizar el particionado para incorporar instancias defectuosas etiquetadas elevó esa métrica a **0.926**. Este hallazgo posee valor metodológico general: reutilizar un conjunto de detección de anomalías en un detector supervisado exige rediseñar las particiones, no solo reutilizar sus directorios.

La tercera conclusión es que el sistema satisface con amplitud el requisito temporal. La latencia media de **12.54 ms**, con un percentil 99 de **20.66 ms**, se mantiene por debajo del umbral de 30 ms establecido, y el rendimiento de **79.8**

FPS ofrece un margen holgado para absorber picos de carga. El componente restrictivo del sistema no es, por tanto, la velocidad, sino el ajuste fino de la localización.

La cuarta conclusión es que el perfil de error observado es el adecuado para un control de calidad industrial. El sistema detecta casi todos los defectos y aprueba correctamente todas las piezas conformes del lote evaluado, mientras que su principal debilidad es la emisión de detecciones redundantes sobre grietas extensas, un error que conserva el rechazo y no compromete la función de control. La quinta conclusión es que la arquitectura por fases favorece la replicabilidad: sustituir la categoría del conjunto de datos basta para adaptar el sistema a otro tipo de pieza.

5.2 Limitaciones

El trabajo presenta limitaciones que condicionan el alcance de sus resultados. La principal es la naturaleza sintética del material de entrenamiento: aunque la réplica reproduce la estructura y el aspecto general de la categoría original, no captura toda la variabilidad de los defectos de fabricación reales, de modo que los valores reportados no son extrapolables sin más a un despliegue productivo. La segunda es que el conjunto de prueba es reducido, lo que otorga a cada instancia un peso elevado y hace que las métricas por clase deban interpretarse con cautela. La tercera es que la representación mediante un único cuadro delimitador resulta insuficiente para defectos de morfología ramificada, como evidencia la precisión de la clase grieta grande. La cuarta es que las mediciones de latencia corresponden a una configuración de hardware concreta y no deben generalizarse a otros equipos.

5.3 Trabajo futuro

Ampliación del conjunto de datos. Incorporar el MVTec AD oficial y, cuando sea posible, imágenes de la línea de producción real, incrementando el número de instancias anotadas por clase hasta el orden de centenares.

Supresión de detecciones redundantes. Ajustar la supresión de no máximos o incorporar un criterio de fusión que agrupe las cajas solapadas sobre una misma

grieta, con el fin de elevar la precisión de la clase grieta grande sin afectar su exhaustividad.

Segmentación en lugar de detección. Explorar modelos de segmentación que aprovechen las máscaras de referencia del conjunto de datos y delimiten los defectos con precisión de píxel, lo que resolvería de raíz la representación de morfologías irregulares.

Calibración del umbral de decisión. Ajustar el umbral de confianza y la lógica de decisión según el costo relativo de los falsos negativos y los falsos positivos en el proceso industrial concreto.

Detección de anomalías no vistas. Complementar el detector supervisado con métodos de detección de anomalías como PatchCore o EfficientAd, capaces de identificar defectos ausentes del conjunto de entrenamiento.

Despliegue en el borde. Validar los modelos exportados en dispositivos periféricos, con el fin de reducir la dependencia de servidores centrales y acercar la inferencia al punto de captura.

Referencias

Bergmann, P., Batzner, K., Fauser, M., Sattlegger, D., & Steger, C. (2021). The MVTec anomaly detection dataset: A comprehensive real-world dataset for unsupervised anomaly detection. *International Journal of Computer Vision, 129*(4), 1038–1059. https://doi.org/10.1007/s11263-020-01400-4

Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). *YOLOv4: Optimal speed and accuracy of object detection*. arXiv. https://doi.org/10.48550/arXiv.2004.10934

Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8* (Versión 8.0) [Software de cómputo]. GitHub. https://github.com/ultralytics/ultralytics

Redmon, J., & Farhadi, A. (2018). *YOLOv3: An incremental improvement*. arXiv. https://doi.org/10.48550/arXiv.1804.02767

Yuan, Y., Xia, Y., & Xu, G. (2025). Metal surface defect detection using SLF-YOLO enhanced YOLOv8 model. *Scientific Reports, 15*, Artículo 94936. https://doi.org/10.1038/s41598-025-94936-9

Repositorio Github

https://github.com/pitsburg8280-cmyk/proyecto_inspeccion_visual.git

Apéndices

Apéndice A. Estructura del repositorio

El proyecto se organiza en módulos independientes que se ejecutan de forma secuencial desde el pipeline principal.

```
proyecto_inspeccion_visual/
├── inspeccion_visual_yolov8.py      Pipeline completo (6 fases)
├── app.py                           Aplicación web de inspección (Streamlit)
├── generar_dataset_sintetico.py     Generación del conjunto de datos
├── generar_figuras_informe.py       Generación de figuras del informe
├── generar_informe_word.py          Generación del documento Word
├── requirements.txt                 Dependencias del proyecto
├── data/
│   ├── mvtec_ad/bottle/             Conjunto de datos (train, test, ground_truth)
│   └── yolo_mvtec/                  Conjunto convertido a formato YOLO
├── models/                          Pesos entrenados (.pt, .onnx, .torchscript)
├── results/                         Métricas, figuras e informes
│   ├── figuras/                     Figuras del documento
│   └── inspections/                 Imágenes de inspección individual
└── runs/                            Registros de entrenamiento de Ultralytics
```

Apéndice B. Reproducibilidad

La totalidad de los resultados presentados puede reproducirse ejecutando los comandos que se enumeran a continuación, en el orden indicado, desde el directorio raíz del proyecto.

Instalación de dependencias:

```
pip install -r requirements.txt
```

Generación del conjunto de datos:

```
python generar_dataset_sintetico.py
```

Ejecución del pipeline completo:

```
python inspeccion_visual_yolov8.py
```

Generación de las figuras:

```
python generar_figuras_informe.py
```

Generación del informe:

```
python generar_informe_word.py
```

Aplicación web de inspección:

```
streamlit run app.py
```

Los artefactos numéricos que sustentan las tablas y figuras se almacenan en formato JSON dentro del directorio results/: evaluation_metrics.json (métricas y desempeño por clase), latency_metrics.json (mediciones de latencia, incluidas las muestras individuales), confusion_matrix.json (matriz de confusión), split_composition.json (composición del conjunto de datos) y batch_inspection_report.json (decisiones de la inspección por lotes). Las semillas de particionado y de entrenamiento se fijan en el código para garantizar la repetibilidad.

Apéndice C. Diccionario de artefactos

Artefactos generados por el pipeline

Archivo	Contenido
dataset_summary.json	Resumen de la categoría y del número de imágenes por clase
split_composition.json	Instancias anotadas por clase en cada partición
training_metrics.json	Duración, épocas y métricas finales del entrenamiento
evaluation_metrics.json	Métricas globales y desagregadas por clase
confusion_matrix.json	Matriz de confusión sobre el conjunto de prueba
latency_metrics.json	Estadísticos de latencia y mediciones individuales
batch_inspection_report.json	Decisión, confianza y latencia por imagen inspeccionada

Nota. Todos los archivos se localizan en el directorio results/. Elaboración propia.